Topic : **Retrieval-Augmented Generation (RAG) Based Intelligent AI System**

Name : Raza Ur Rahman

Date : 25 Feb 2026

Organization : 10xGrowth

---

1. Abstract

This project presents the design and implementation of a Retrieval-Augmented Generation (RAG) based Intelligent Question Answering System. The system integrates a Large Language Model (LLM) with a structured document-based knowledge base to generate accurate, context-aware responses.

Traditional LLM systems rely solely on pre-trained knowledge, which may result in hallucinated or outdated answers. The proposed RAG architecture addresses this issue by retrieving relevant information from uploaded documents before generating responses. This ensures that outputs are grounded in actual document content, improving reliability and transparency.

---

2. Introduction

Large Language Models (LLMs) such as GPT-based systems are capable of generating high-quality human-like responses. However, these models have inherent limitations:

- They do not have real-time access to external data.
- They may hallucinate facts.
- They cannot directly reference domain-specific documents unless fine-tuned.

To overcome these limitations, Retrieval-Augmented Generation (RAG) combines semantic search with generative AI. Instead of directly answering a query, the system first retrieves relevant information from a document repository and then passes that information to the LLM to generate a grounded response.

This project implements a complete RAG pipeline using FastAPI, FAISS, HuggingFace embeddings, LangChain, and OpenRouter LLM integration.

---

3. Problem Statement

Standalone LLM systems generate responses based on pre-trained parameters without verifying external knowledge sources. This may result in:

- Hallucinated information
- Inaccurate responses
- Lack of domain grounding
- No source transparency

The objective of this project is to build a RAG-based AI system that:

- Retrieves relevant document content
- Uses semantic similarity search
- Augments LLM input with retrieved context
- Produces grounded and reliable answers
- Displays document sources for transparency

---

4. System Architecture

4.1 Architecture Overview

The system follows a Retrieval-Augmented Generation architecture. Instead of generating answers directly from the LLM, it first performs semantic retrieval from a vector database.

This improves factual grounding and reduces hallucination.

4.2 Architectural Workflow

The system operates in the following stages:

Step 1 – User Query

The user submits a question through the web interface.

Step 2 – Query Embedding Generation

The query is converted into a dense vector embedding using a pre-trained embedding model (MiniLM).

Step 3 – Vector Similarity Search

The query embedding is compared with stored document embeddings in a FAISS vector database. The top-K most similar document chunks are retrieved.

Step 4 – Context Augmentation

The retrieved document chunks are combined and passed as contextual input to the LLM.

Step 5 – LLM Response Generation

The LLM generates a response grounded in the retrieved context.

Step 6 – Source Display

The system displays the generated answer along with document source references.

4.3 Architecture Diagram

You can insert this diagram in your PDF:

```
User Query
↓
Embedding Model (MiniLM)
↓
FAISS Vector Database
↓
Top-K Relevant Chunks
↓
LLM (OpenRouter API)
↓
Final Context-Aware Response
```

---

5. Technology Stack

5.1 Backend – FastAPI

FastAPI was used to build the backend API due to its high performance and lightweight structure. It allows easy integration with AI libraries and efficient request handling.

5.2 Vector Database – FAISS

FAISS (Facebook AI Similarity Search) was used to store and retrieve vector embeddings efficiently. It enables fast nearest-neighbor search, which is essential for semantic retrieval.

5.3 Embedding Model – HuggingFace MiniLM

The all-MiniLM-L6-v2 model was used to generate dense vector embeddings for both document chunks and user queries. This model provides strong semantic understanding while maintaining computational efficiency.

5.4 LLM Integration – OpenRouter API

The OpenRouter API was used to integrate LLM capabilities into the system. Models such as GPT-4o-mini or Llama-3-8B-Instruct were used to generate context-aware responses.

5.5 Orchestration – LangChain

LangChain was used to structure and simplify the RAG pipeline, including:

- Document loading
- Text splitting
- Embedding generation
- Retrieval
- LLM chaining

5.6 Frontend Interface

A basic HTML/CSS interface was implemented to allow user interaction. The interface enables question submission and displays answers along with source documents.

---

6. Implementation Details

6.1 Document Ingestion

PDF documents are placed inside the docs/ folder. The system loads these documents using PyPDFLoader and extracts text content.

6.2 Text Chunking

Extracted text is divided into smaller semantic chunks using RecursiveCharacterTextSplitter. This improves retrieval accuracy.

6.3 Embedding Generation

Each text chunk is converted into a numerical vector representation using the MiniLM embedding model.

6.4 Vector Storage

All embeddings are stored in FAISS, which enables efficient similarity search.

6.5 Query Processing

When a user submits a query:

- The query is embedded.
- Similar document chunks are retrieved.
- Retrieved content is passed to the LLM.
- Final answer is generated.

---

7. Evaluation Results

7.1 Retrieval Effectiveness

The system was tested using multiple domain-specific queries. In most cases, relevant document chunks were successfully retrieved before response generation.

The displayed document sources confirmed that responses were grounded in actual document content.

7.2 Hallucination Reduction

Out-of-scope queries (not related to uploaded documents) were tested. The system restricted responses to available document context, demonstrating reduced hallucination compared to standalone LLM behavior.

7.3 Response Performance

The average response time observed during testing ranged between 1.5–3 seconds per query. This includes embedding generation, similarity search, context retrieval, and LLM response generation.

The system demonstrates near real-time usability.

---

8. Challenges Faced

- Dependency installation errors
- API configuration issues
- Initial embedding model download delay
- Port conflicts during server startup

These issues were resolved through debugging and environment configuration.

---

9. Conclusion

The project successfully implemented a Retrieval-Augmented Generation based AI system that integrates document retrieval with LLM-based response generation.

The system:

- Produces context-aware answers
- Reduces hallucination
- Provides source transparency
- Meets the stated project objectives

This implementation fulfills all requirements specified in the project expectations.

---

10. Future Enhancements

- Persistent FAISS index storage
- Conversational chat-style interface
- Hybrid search (vector + keyword)
- Cloud deployment
- Advanced evaluation metrics